MINI PROJECT REPORT
ON
2D Image Puzzle Game

submitted in partial fulfillment of requirements
for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

by

M.DURGA	**(229Y1A0575)**
M.JUHA FATHIMA	**(229Y1A0576)**
M.NAVYA SREE	**(229Y1A0574)**
M.VENKATA CHAITHANYA	**(229Y1A0577)**
M.VIJAYA	**(229Y1A0578)**

under the Esteemed Supervision of

Mrs.B.Manorama Devi, MTech., (Ph.D).
Assistant Professor.,
Dept of CSE

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

K.S.R.M. COLLEGE OF ENGINEERING

(UGC-Autonomous)
Approved by AICTE , New Delhi and affiliated to JNTUA, Anantapuramu
Accredited by NAAC with A+Grade &B.tech (EEE,ECE,CSE,CE and ME) Programs by NBA
Kadapa, Andhra Pradesh, India– 516 003

2024-2025

K.S.R.M. COLLEGE OF ENGINEERING

(UGC-Autonomous)
Approved by AICTE , New Delhi and affiliated to JNTUA, Anantapuramu
Accredited by NAAC with A+Grade &B.tech (EEE,ECE,CSE,CE and ME) Programs by NBA
Kadapa, Andhra Pradesh, India– 516 003

VISION

To evolve as center of repute for providing quality academic programs amalgamated with creative learning and research excellence to produce graduates with leadership qualities, ethical and human values to serve the nation.

MISSION

M1: To provide high quality education with enriched curriculum blended with impactful teaching-learning practices.

M2: To promote research, entrepreneurship and innovation through industry collaborations.

M3: To produce highly competent professional leaders for contributing to Socio-economic development of region and the nation.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VISION

To evolve as a recognized center of excellence in the area of Computer Science and Engineering and other related inter-disciplinary fields.

MISSION

M1: To produce competent and industry ready professionals through well balanced curriculum and innovative pedagogy.

M2: To provide conducive environment for research by establishing centre of excellence and industry collaborations.

M3: To install leadership qualities, ethical values among students through various co-curricular and extracurricular activities.

B. Tech. (COMPUTER SCIENCE AND Engineering)

PROGRAM EDUCATIONAL OBJECTIVES

B. Tech-Computer Science and Engineering Program Objectives.

A graduate of the K.S.R.M.C.E, C.S.E should have a successful career in CSE or a related field, and within three to five years, should

PEO1: To excel in their career as competent software engineer in IT and allied organizations.

PEO2: To pursue higher education and to demonstrate research temper for providing solutions to engineering problems.

PEO3: To contribute for the societal development by exhibiting leadership, through professional, social and ethical values.

PROGRAM OUTCOMES

PO1 - Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2 - Problem Analysis: Identify, formulate, review research literature, and analyse complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3 - Design/Development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4 - Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5 - Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and

modelling to complex engineering activities with an understanding of the limitations.

PO6 - The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7 - Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8 - Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of engineering practice.

PO9 - Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10 - Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11 - Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12 - Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES

PSOs are the statements that describes what the graduates of a specific engineering program should be able to:

PSO1-Professional Skills: The ability to understand, analyse and develop the computer programs in areas related to algorithms, system software, multimedia, web design, big data analysis and networking for efficient design of computer-based systems of varying complexity.

PSO2-Problem Solving Skills: The ability to apply standard practices and strategies in software project development using open – ended programming environments to deliver a quality product or business success

PSO3 – Successful career and Entrepreneurship: The ability to employ modern computer languages, environments, and platforms in creating innovative career paths to be an entrepreneur, and Zest for higher studies

COURSE OUTCOMES

CO1. Prove the correctness and analyze space and time complexity of an algorithm.

CO2. Understand different algorithm design strategies..

CO3. Analyze & Apply standard algorithms.

CO4. Understand Graph/Tree bases applications and appropriate techniques.

CO5. Current trends in Non Deterministic concepts.

CO-PO MAPPING

Course Outcome	Program Outcomes												Program Specific Outcomes		
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO 10	PO 11	PO 12	PSO1	PSO2	PSO3
CO1	3	3											3	3	
CO2			3	3									3		3
CO3					3	3							2	2	3
CO4									3	3			1		
CO5							3	3			3		2	2	

K.S.R.M. COLLEGE OF ENGINEERING

(UGC-Autonomous)
Approved by AICTE , New Delhi and affiliated to JNTUA, Anantapuramu
Accredited by NAAC with A+Grade &B.tech (EEE,ECE,CSE,CE and ME) Programs by NBA
Kadapa, Andhra Pradesh, India– 516 003

CERTIFICATE

This is to certify that the Design and Analysis of Algorithms Mini project entitled

2D Image puzzle Game

is the bonafide work done & submitted by

M.DURGA	**(229Y1A0575)**
M.JUHA FATHIMA	**(229Y1A0576)**
M.NAVYA SREE	**(229Y1A0574)**
M.VENKATA CHAITHANYA	**(229Y1A0577)**
M.VIJAYA	**(229Y1A0578)**

in the Department of Computer Science and Engineering,

K.S.R.M.C.E, Kadapa and is submitted to **Jawaharlal Nehru Technological University Anantapur,** in partial fulfilment of the requirements for the award of degree of Bachelor of Technology in Computer Science and Engineering during 2022-2026.

Supervisor	**Internal Examiner**	**Head of the Department**
Mrs.B.Manorama Devi **M.Tech,(Ph.D).** ***Assistant Professor*** ***Dept of CSE.***		**A. Ram Prakash Reddy** **M.Tech,(Ph.D).** **Assistant Professor & HOD** ***Dept of CSE.***

DECLARATION

We hereby declare that this Mini Project report titled ""**2D Image Puzzle game**" is a genuine project work carried out by us, in B. Tech (**Computer Science and Engineering**) degree course of Jawaharlal Nehru Technological University Anantapur and has not been submitted to any other course or University for the award of any degree by us.

Signature of the Student

M.Navya sree

M.Durga

M.Juha Fathima

M.Venkata chaithanya

M.vijaya

ACKNOWLEDGEMENTS

An endeavor over a long period can be successful only with the advice and supports of many well-wishers. We take this opportunity to express our gratitude and appreciation to all of them.

We are extremely thankful to our beloved Managing Director **Dr. K Chandra Obul Reddy** Garu who took keen interest and encouraged us in every effort throughout this course. We are deeply indebted to the supervisor, **Mrs.B.Manorama Devi**, M.Tech.,(Ph.D)., Assistant Professor, Department of Computer Science and Engineering for valuable guidance, constructive criticism and keen interest evinced throughout the course of our DAA project work. We are really fortunate to associate ourselves with such an advising and helping guide in every possible way, at all stages, for the successful completion of this work.

We express our deep sense of gratitude to **A.Ram Prakash Reddy**, M.Tech., (Ph.D)., Assistant Professor and Head of Department of Computer Science and Engineering for his valuable guidance and constant encouragement given to us during this DAA project and the course.We take this opportunity to express our deep gratitude and appreciation to all those who encouraged us for successfully completion of this Community Service Project work. We wish to express our sincere to gratitude to **Dr. T NAGESWARA PRASAD**, M.Tech., Ph.D. Vice Principal of K.S.R.M.C.E, Kadapa and **Dr. V.S.S. MURTHY**, M.Tech., Ph.D. Principal of K.S.R.M.C.E, Kadapa and for their consistent help and encouragement to complete the DAA project work. We are pleased to express our heart full thanks to our faculty in Department of CSE of KSRMCE for their moral support and good wishes. Finally, we have a notation to express our sincere thanks to friends and all those who guided, inspired and helped us in the completion of our project work.

M.Durga(229Y1A0575)

M.Juha Faithama(229Y1A0576)

M.Navya sree(229Y1A0574)

M.venkata chaithanya(229Y1A0577)

M.vijaya(229Y1A0578)

INDEX

ABSTRACT 1
1.Introduction 2
2.MODULES 3
2.1. main: 3
2.2 Divide and Conquer 3
2.3.Implementation Details: 4
2.4. How The Image Is Split And Combined: 6
3.PROJECT PICTORIAL REPRESENTATION 9
4.SOURCE CODE 11
5.INPUT AND OUTPUT SCREENS 12
CONCLUSION 25
REFERENCES 26

ABSTRACT

Puzzle games have made their mark in the world of digital entertainment and mental growth. They not only keep players engaged but also help sharpen their problem-solving skills and thinking abilities. This project centers on creating a 2D image puzzle game aimed at offering a fun and interactive user experience. Our main goal is to craft a game that entertains while also pushing players to think harder through puzzles that get trickier as they go.

The design of the 2D image puzzle game is made to be easy to understand and user-friendly, appealing to a wide range of players, from kids to adults. It follows the classic sliding puzzle style, where players need to rearrange mixed-up images to make a complete picture. We used a mix of programming and graphic design to create a game that's both visually nice and functional. The game runs on a solid game engine that supports 2D graphics, ensuring smooth animations and responsive controls

The game dynamically slices a single large image into smaller 100px x 100px pieces using CSS properties and JavaScript.

The outcome of this project shows how effective the 2D image puzzle game is at engaging players while supporting skills like pattern recognition, spatial awareness, and logical reasoning. Players expressed satisfaction with the game’s design, mechanics, and overall enjoyment. It serves not only as a source of fun but also as a tool for mental growth, making it useful for educational settings.

1.INTRODUCTION

To A 2D image puzzle game is a game where players drag and drop broken pieces of an image to form the correct picture. Sliding puzzles are essentially two-dimensional in nature, even if the sliding is facilitated by mechanically interlinked pieces (like partially encaged marbles) or three-dimensional tokens. A sliding puzzle, sliding block puzzle, or sliding tile puzzle is a combination puzzle that challenges a player to slide (frequently flat) pieces along certain routes (usually on a board) to establish a certain end-configuration. The pieces to be moved may consist of simple shapes, or they may be imprinted with colors, patterns, sections of a larger picture (like a jigsaw puzzle), numbers, or letters.

To make our idea real, we paid attention to how the game looks and feels. We aimed for a design that is nice to look at and easy to use. The interface is made to be simple, so players of all ages can enjoy it. Bright graphics and animations make the game more attractive. Plus, players can pick images from a gallery or use their own, adding a personal touch that keeps them coming back. This feature not only makes the game more fun but also lets players challenge themselves with different images and levels. The game mechanics are easy to understand but still offer a challenge. Players need to think carefully about how to move the pieces on the sliding puzzle board. A timer and scoring system add a competitive twist, encouraging players to get better and finish puzzles quickly. This game element makes it more enjoyable and addictive.

The 2D image puzzle is often used as a brain-teasing activity that improves problem-solving skills, spatial awareness, and cognitive abilities. In the game, players can visually track their progress and are usually given a time limit or a move counter to add an element of challenge. The game may also feature different difficulty levels, with varying grid sizes and complexity based on the number of pieces

2.MODULES

2.1. main:

The game dynamically slices a single large image into smaller 100px x 100px pieces using CSS properties and JavaScript. The key to breaking the image lies in leveraging the background-position property for each piece, ensuring only a specific portion of the image is visible in each puzzle piece. Then, the game randomly shuffles these pieces and places them in the puzzle grid for the player to solve.

Here we use an algorithm for breaking the into pieces. The divide and conquer algorithm also take the breaking pieces and form the puzzle.

2.2 Divide and Conquer:

A divide and conquer algorithm is a strategy of solving a large problem by

1. breaking the problem into smaller sub-problems
2. solving the sub-problems, and
3. combining them to get the desired output.

2.2.1 Divide and Conquer Strategy:

The Divide and Conquer approach is applied to shuffle the puzzle pieces. The image is divided into smaller segments (e.g., 3x3 tiles), each shuffled independently. This method reduces the complexity of shuffling large datasets and ensures randomized yet solvable configurations.

2.2.2 How Divide and Conquer Algorithms Work?

Here are the steps involved:

1. **Divide**: Divide the given problem into sub-problems using recursion.
2. **Conquer**: Solve the smaller sub-problems recursively. If the subproblem is small enough, then solve it directly.
3. **Combine:** Combine the solutions of the sub-problems that are part of the recursive process to solve the actual problem

2.2.3 Pseudocode:

```
Divide(image):
    Split image into N segments
    Shuffle each segment independently
    Merge shuffled segments into final grid
```

2.3 Implementation Details:

Frontend: Developed using HTML, CSS, and JavaScript.

Algorithmic Optimization: JavaScript functions are optimized for better performance using Divide and Conquer principles.

1. HTML Structure

```
<div id="game-container">
   <h1 class="gameName">2D Image Puzzle Game</h1>
   <div id="timer">Time Left: 60s</div>
   <div id="puzzle-container"></div>
   <button id="submit-btn">Submit</button>
   <button id="restart-btn">Play Again</button>
</div>
```

2. CSS Styling and JavaScript Logic

```
body {
    background: url('back.avif') center/cover no-repeat;
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    height: 100vh;
}
.puzzle-piece {
    width: 100px;
    height: 100px;
    cursor: pointer;
    transition: all 0.3s ease-in-out;
}
```

```
const gridSize = 3;
let pieces = [];
function initializePuzzle() {
    let image = getRandomImage();
    divideImage(image);
    shufflePieces();
    renderPuzzle();
}
function divideImage(image) {
    for (let i = 0; i < gridSize * gridSize; i++) {
        let piece = document.createElement('div');
        piece.style.backgroundImage = url(${image});
        pieces.push(piece);
    }
```

2.4 How the Image is Split and Combined:

2.4.1 Setting Up the Puzzle Grid:

- The HTML structure uses a div with an ID of puzzle-container. This container is styled to display as a grid with a 3x3 layout, where each cell in the grid is 100px by 100px.
- The style defined in the CSS section:

CSS:

```
#puzzle-container {
display: grid;
grid-template-columns: repeat(3, 100px);
grid-template-rows: repeat(3, 100px);
gap: 1px;
border: 2px solid #333;
margin-bottom: 20px;
}
```

2.4.2 Loading the Image:

- The game randomly selects an image from a predefined list of URLs using the getRandomImage()

Javascript:

function:

```
function getRandomImage() {
   return imageUrls[Math.floor(Math.random() * imageUrls.length)];
}
```

2.4.3 Creating Puzzle Pieces:

- The chosen image is set as the background for each puzzle piece, using the background-image property.
- For each of the **9 pieces** in the 3x3 grid, a new div element is created and styled to show only a specific section of the larger image.

- Each puzzle piece is assigned a background-position that shifts the view to different parts of the original image. Here's how this works:

Javascript:

```
for (let i = 0; i < gridSize * gridSize; i++) {
    let piece = document.createElement('div');
    piece.classList.add('puzzle-piece');
    piece.style.backgroundImage = `url(${currentImage})`;
    piece.style.backgroundPosition = `${-(i % gridSize) * 100}px ${-Math.floor(i /
gridSize) * 100}px`;
    piece.setAttribute('data-index', i);
    pieces.push(piece);
}
```

The formula used to set background-position:

- **X-axis**: -(i % gridSize) * 100px
    - This positions the image horizontally by moving it to the left, effectively showing the correct part of the image for each column.
- **Y-axis**: -Math.floor(i / gridSize) * 100px
    - This positions the image vertically by moving it up, showing the correct part of the image for each row.

2.4.4 Shuffling the Pieces:

- Once all pieces are created, they are shuffled using a custom **Divide and Conquer** shuffling algorithm.
- The shuffled pieces are then appended to the puzzle-container.

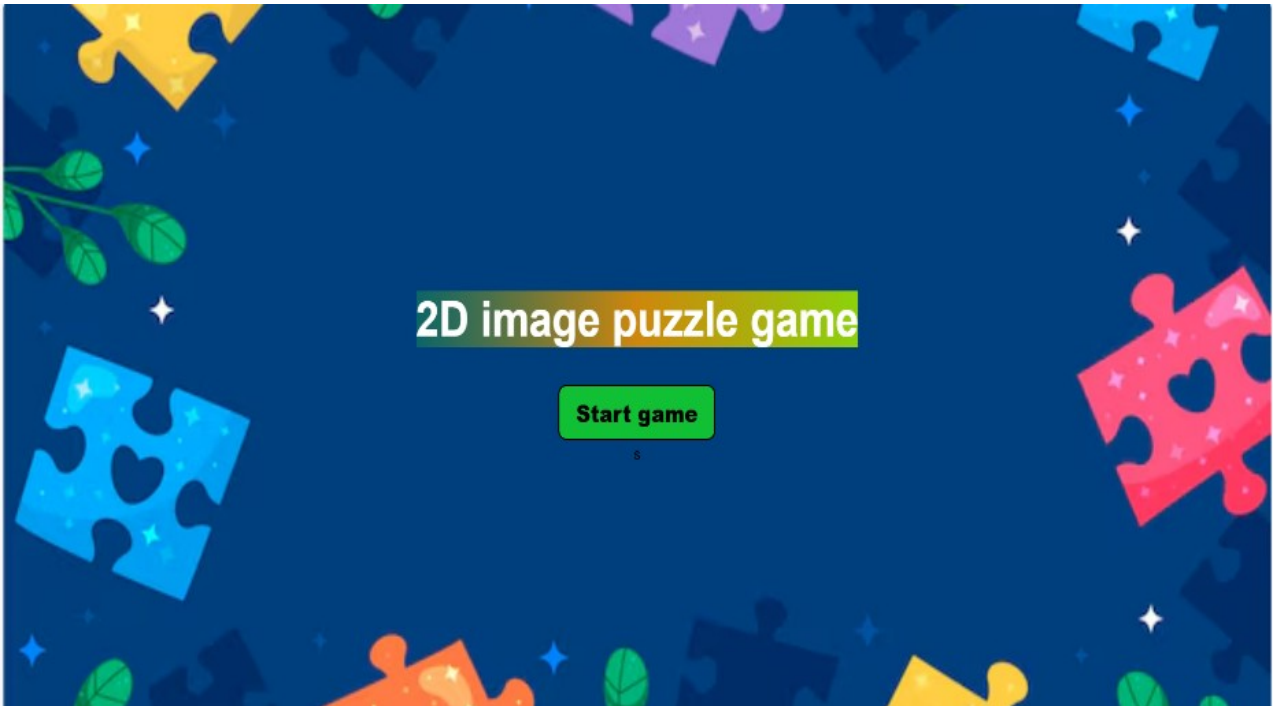
```
function divideAndConquerShuffle(arr) {
    f (arr.length <= 1) return arr;
     const mid = Math.floor(arr.length / 2);
     const left = divideAndConquerShuffle(arr.slice(0, mid));
      const right = divideAndConquerShuffle(arr.slice(mid));
      return merge(left, right);
    }
```

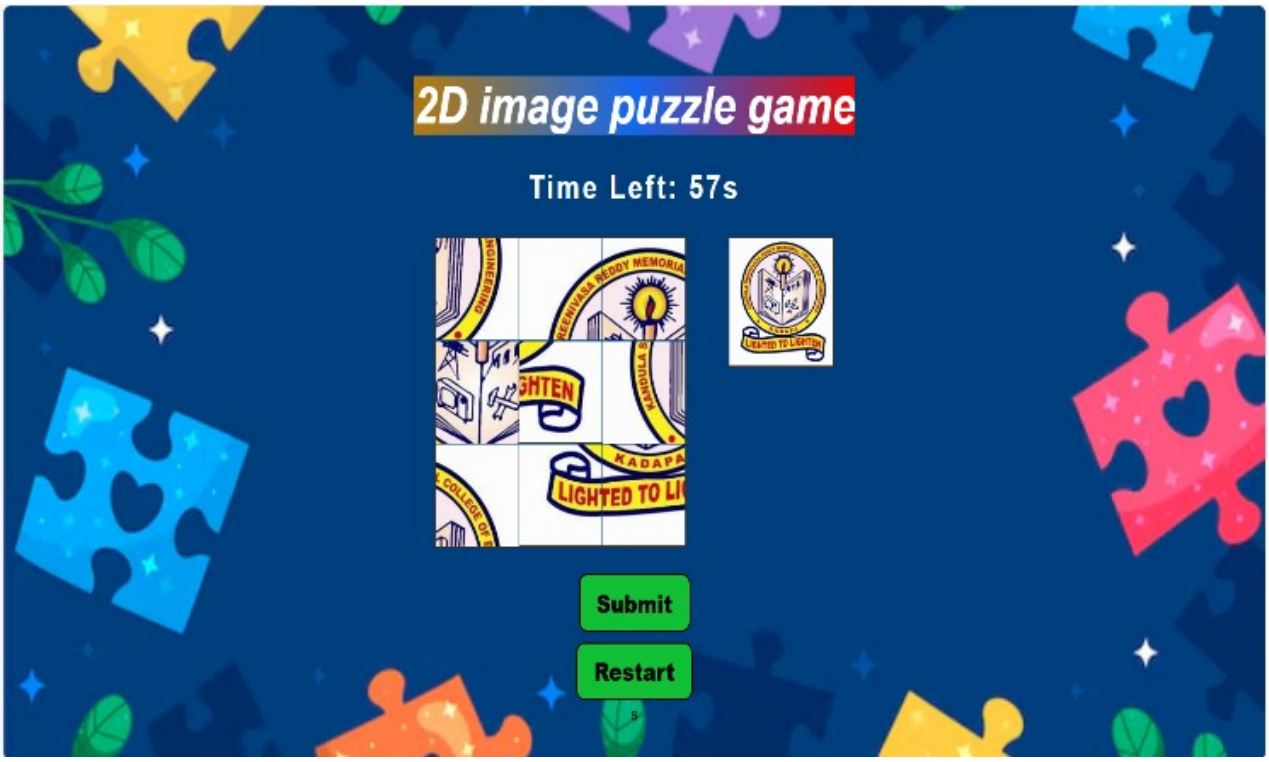
```
function merge(left, right) {
 let result = [];
while (left.length && right.length) {
if (Math.random() < 0.5) {
        result.push(left.shift());
} else {
        result.push(right.shift());
      }
  }
  return [...result, ...left, ...right];
}
```

2.4.5 Interactivity:

- Users can click on the pieces to swap them. The game keeps track of the number of moves and shows a timer for 60 seconds.
- If the player correctly arranges all the pieces back into their original positions within the time limit, they win the game.

3. PROJECT PICTURAL REPRASENTATION




2D image puzzle game
Time Left: 57s
Submit
Restart

4.SOURCE CODE

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>2D Image Puzzle Game</title>
   <style>
      body {
         display: flex;
         background-
image:url("https://assets.onecompiler.app/42xeuatdq/42xstgz6g/back.avif");
         flex-direction: column;
         align-items: center;
         justify-content: center;
         height: 100vh;
        background-size: cover;
        background-position: center;
         margin: 0;
         font-family: Arial, sans-serif;
         overflow: hidden;
      }
      #game-container {
         display: flex;
         align-items: flex-start;
         gap: 50px;
         display: none;
      }
```

```
#puzzle-container {
        display: grid;
        grid-template-columns: repeat(3, 100px);
        grid-template-rows: repeat(3, 100px);
        gap: 1px;
        border: 2px solid #333;
        margin-bottom: 20px;
    }
    .puzzle-piece {
        width: 100px;
        height: 100px;
        background-size: 300px 300px;
        cursor: pointer;
        transition: border 0.3s;
    }
    .selected {
        border: 2px solid red;
    }
    #timer,#moves {
        font-size: 32px;
        margin-bottom: 30px;
        color: white;
        font-weight: bold;
        text-transform: capitalize;
        letter-spacing: 3px;
        display: none;
    }
```

```
#submit-btn,#restart-btn{
        padding: 10px 20px;
        font-size: 16px;
        cursor: pointer;
        font-size: 24px;
        font-weight: 800;
        border-radius: 10px;
        border-style: solid;
        margin: 5px;
        background-color: rgb(17, 192, 52);
        display: none;
    }
    #play{
        padding: 10px 20px;
        font-size: 16px;
        cursor: pointer;
        font-size: 24px;
        font-weight: 800;
        border-radius: 10px;
        border-style: solid;
        margin: 5px;
        background-color: rgb(17, 192, 52);
        animation-name:start;
        animation-iteration-count:1;
        animation-fill-mode: both;
        animation-duration: 3s;
    }
```

```
 #original-image {
        width: 125px;
        height: 125px;
        border: 2px solid #333;
        background-size: 100% 100%;
        display: none;
    }
    .gameName {
        font-size: 50px;
        background-image: linear-gradient(to left,rgb(191, 11, 11),rgb(0, 98, 255),orange);
         border: none;
        color: white;
        font-style: oblique;
        border-radius: c;
        width: fit-content;
        animation-name: name;
        animation-iteration-count: infinite;
        animation-fill-mode: both;
        animation-duration: 3s;
    }
@keyframes name {
        0% {
            background-image:linear-gradient(to left,rgb(70, 153, 7),rgb(143, 12,
147),rgb(9, 129, 133));
            font-style: italic;
        }
50% {
            background-image:linear-gradient(to left,rgb(141, 211, 12),rgb(207, 136,
14),rgb(11, 102, 105));
            font-style: normal;
}
```

```
100% {
            background-image:linear-gradient(to left,rgb(230, 13, 13),rgb(19, 104,
241),rgb(184, 123, 10));
            font-style: italic;
        }
    }
    @keyframes start {
        0% {
            font-size: 50px;
            font-style: italic;
        }
        50% {
            rotate:360deg;
            font-style: oblique;
            filter: hue-rotate(720deg);
        }
        100% {
           font-style: bold;
        }
    }
    @media (max-width:767px) {
        #game-container {
            gap: 5px;
        }
   .gameName {
            font-size:40px ;
        }
    }
   </style>
</head>
```

```
<body>
    <h1 class="gameName">2D image puzzle game</h1>
    <div id="start-game">
        <button id="play">Start game</button>
    </div>
    <div id="timer">Time: 60s</div>
    <div id="moves">Moves: 0</div>
    <div id="game-container">
        <div id="puzzle-container"></div>
        <div id="original-image"></div>
    </div>
    <button id="submit-btn">Submit</button>
    <button id="restart-btn">Restart</button>

    <script>
        const startGame = document.getElementById("start-game");
        const play = document.getElementById("play");
        const gameContainer = document.getElementById("game-container");
        const container = document.getElementById("puzzle-container");
        const submitBtn = document.getElementById("submit-btn");
        const restartBtn = document.getElementById("restart-btn");
        const timerDisplay = document.getElementById("timer");
        const movesDisplay = document.getElementById("moves");
        const originalImageDisplay = document.getElementById("original-image");
        const gridSize = 3;
        let pieces = [];
        let selectedPiece = null;
        let currentImage = '';
        let timerInterval;
```

```
let moves = 0;
let gameOver = false;
const imageUrls = [
    'https://assets.onecompiler.app/42rkv658c/42xdyqv5t/1000038909.jpg',
    'https://assets.onecompiler.app/42rkv658c/42xdyqv5t/1000037454.jpg',
    'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/animal.png',
    'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/bheem.jpg',
    'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/bird.webp',
      'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/bujji.jpg',
      'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/Dholu_Bholu.webp',
      'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/dhoni.jpeg',
      'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/dhoni1.avif',
      'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/dora.png',
      'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/dq.jpg',
      'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/forest.jpg',
      'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/heidi.jpg',
      'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/heidi.jpg',
      'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/karthik.jpeg',
      'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/keerthi.jpeg',
      'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/krish.jpg',
      'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/krishna.jpg',
      'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/md.jpg',
      'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/motu.webp',
      'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/nobita.webp',
      'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/nobithad.jpg',
    'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/patlu.jpeg',
      'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/prabhas.webp',
      'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/purple.jpg',
```

```
    'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/ram2.avif',
     'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/ramcharan.jpeg',
     'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/ratan.webp',
     'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/saipallavi.jpg',
     'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/shinchan.webp',
     'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/smalltom.jpg',
     'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/tom.jpg',
     'https://assets.onecompiler.app/42xeuatdq/42xstgz6g/chutki.webp',
];
 // Function to get a random image
 function getRandomImage() {
    return imageUrls[Math.floor(Math.random() * imageUrls.length)];
 }
 play.addEventListener("click",()=> {
    play.style.display ="none";
    gameContainer.style.display ="flex";
    originalImageDisplay.style.display = "flex";
    timerDisplay.style.display = "block";
    submitBtn.style.display = "flex";
    movesDisplay.style.display = "block";
    restartBtn.style.display = "flex";
    clearInterval(timerInterval);
    timeRemaining = 60;
    gameOver = false;
    timerDisplay.textContent = 'Time Left: 60s';
    startTimer();
 })
```

```
// Divide and Conquer Shuffle
    function divideAndConquerShuffle(arr) {
        if (arr.length <= 1) return arr;
        const mid = Math.floor(arr.length / 2);
        const left = divideAndConquerShuffle(arr.slice(0, mid));
        const right = divideAndConquerShuffle(arr.slice(mid));
        return merge(left, right);
    }
    function merge(left, right) {
        let result = [];
        while (left.length && right.length) {
            if (Math.random() < 0.5) {
                result.push(left.shift());
            } else {
                result.push(right.shift());
            }
        }
        return [...result, ...left, ...right];
    }
// Function to initialize the puzzle
    function initializePuzzle() {
        clearInterval(timerInterval);
        timeRemaining = 60;
        moves = 0;
        gameOver = false;
        timerDisplay.textContent = 'Time Left: 60s';
        movesDisplay.textContent = `Moves: ${moves}`;
        startTimer();
```

```
        container.innerHTML = ''; // Clear existing pieces
        pieces = [];
        currentImage = getRandomImage();
        originalImageDisplay.style.backgroundImage = `url(${currentImage})`;
        for (let i = 0; i < gridSize * gridSize; i++) {
            let piece = document.createElement('div');
            piece.classList.add('puzzle-piece');
            piece.style.backgroundImage = `url(${currentImage})`;
            piece.style.backgroundPosition = `${-(i % gridSize) * 100}px ${-Math.floor(i /
gridSize) *        100}px`;
            piece.setAttribute('data-index', i);
            pieces.push(piece);
// Shuffle using Divide and Conquer method
        pieces = divideAndConquerShuffle(pieces);
        // Append pieces to the container
        pieces.forEach(piece => container.appendChild(piece));
        // Add click event listeners for swapping
        pieces.forEach(piece => {
            piece.addEventListener('click', () => handlePieceClick(piece));
        });
    }
// Function to handle piece clicks
    function handlePieceClick(piece) {
        if (gameOver) return; // Prevent interaction when game is over

        if (!selectedPiece) {
            selectedPiece = piece;
            piece.classList.add('selected');
        }
```

```
else{
        swapPieces(selectedPiece, piece);
            selectedPiece.classList.remove('selected');
            selectedPiece = null;
            moves++;
            movesDisplay.textContent = `Moves: ${moves}`;
        }
    }
// Function to swap two pieces
    function swapPieces(piece1, piece2) {
        const tempIndex = piece1.style.backgroundPosition;
        piece1.style.backgroundPosition = piece2.style.backgroundPosition;
        piece2.style.backgroundPosition = tempIndex;
    }
// Function to check if the puzzle is solved
    function isSolved() {
        return Array.from(container.children).every((piece, index) => {
            return piece.style.backgroundPosition ===
                `${-(index % gridSize) * 100}px ${-Math.floor(index / gridSize) * 100}px`;
        });
    }
// Timer function with countdown and game over check
    function startTimer() {
        timerInterval = setInterval(() => {
            timeRemaining--;
            timerDisplay.textContent = `Time Left: ${timeRemaining}s`;

            if (timeRemaining <= 0) {
                clearInterval(timerInterval);
                gameOver = true; // Set game over to true
```

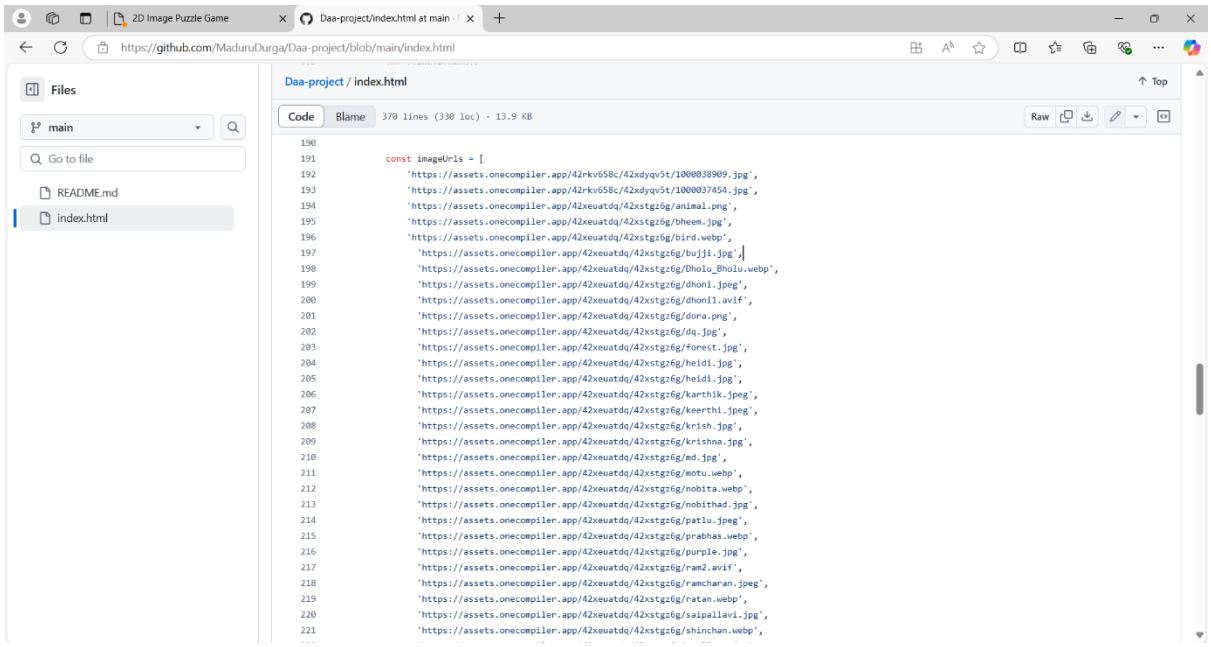
```
            alert("Time's up! You didn't solve the puzzle in time. Game over.");
            }
        }, 1000);
    }
  // Event listener for the submit button
    submitBtn.addEventListener('click', () => {
        if (gameOver) {
            alert("The game is over. Restart to play again.");
            return;
        }
if (isSolved()) {
            clearInterval(timerInterval);
            alert(`Congratulations! You solved the puzzle with ${moves} moves and
${timeRemaining} seconds remaining.`);
        } else {
            alert("The puzzle is not solved yet. Keep trying!");
        }
    });

    // Event listener for the restart button
    restartBtn.addEventListener('click', initializePuzzle);

    // Initialize the puzzle on page load
    initializePuzzle();
  </script>
</body>
</html>
```

5.INPUT AND OUTPUT SCREENS

INPUTS

The 2D puzzle game uses a set of pre-defined images as input, which serve as the source for creating the puzzle pieces. These images are selected randomly from a diverse collection, including various types of pictures such as animals, famous personalities, cartoons, and landscapes.

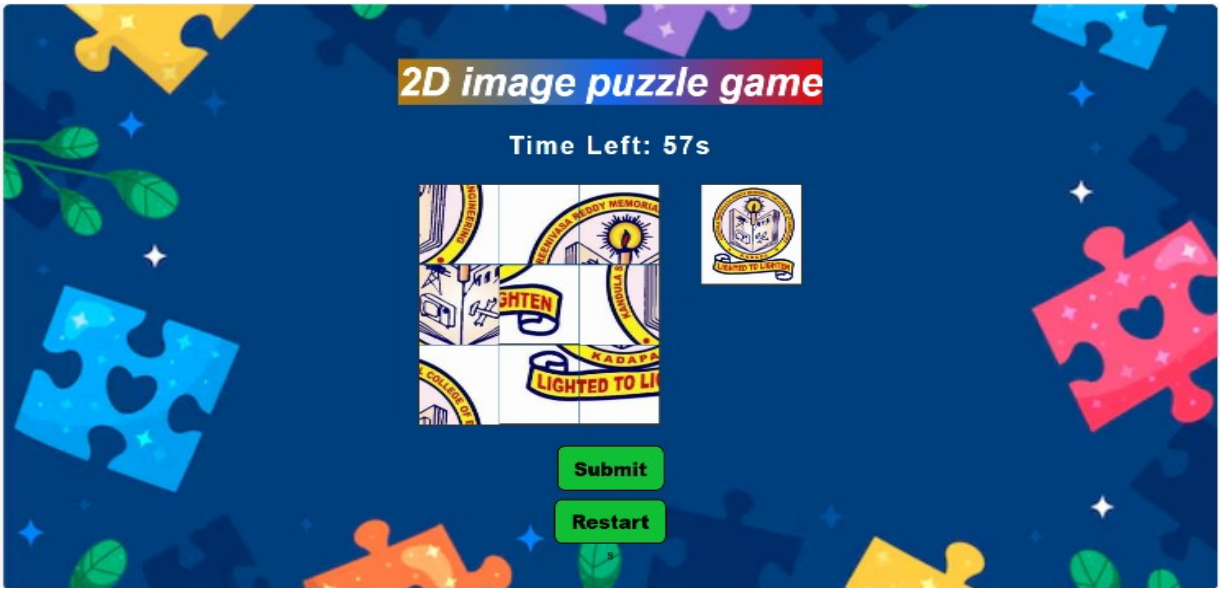
These images are loaded dynamically when the game starts, ensuring a unique experience with every playthrough.

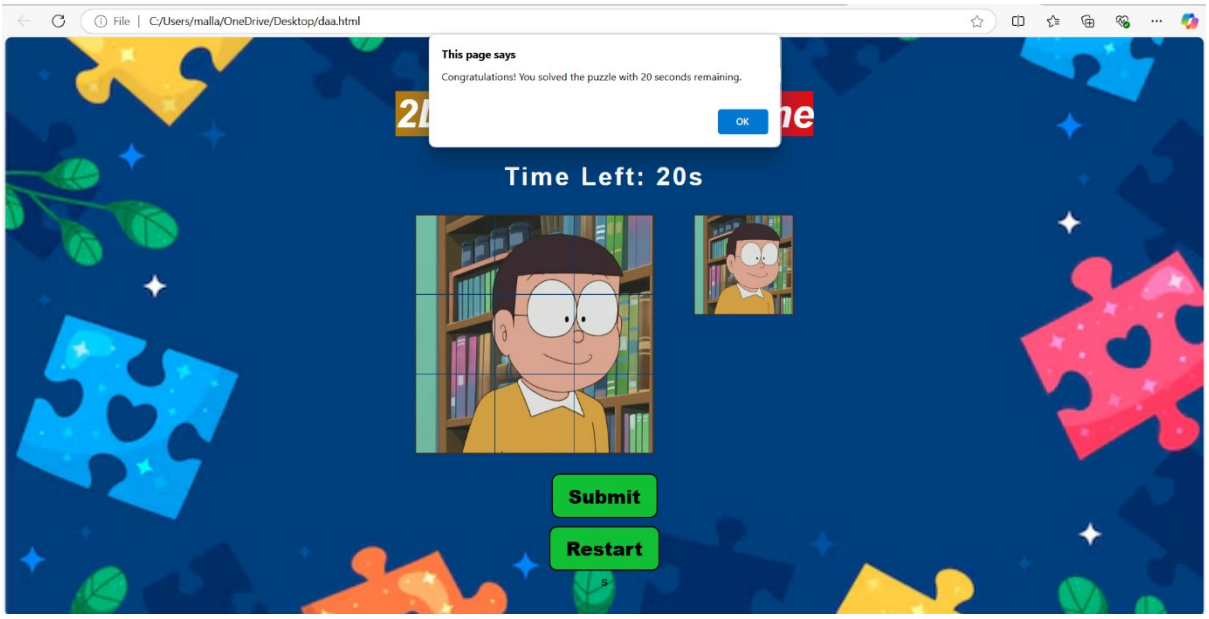


OUTPUT

The output of the 2D puzzle game includes the visual display of the puzzle grid, where users can interact with individual pieces to solve the puzzle. As players swap pieces, the game provides real-time feedback through the update of the puzzle layout. Additional outputs include the display of a countdown timer showing the remaining time, and a move counter that tracks the number of swaps made by the player. Upon completion or time expiration, the game outputs a message indicating whether the player has successfully solved the puzzle, including the number of

moves made and the time remaining. Furthermore, users receive feedback through buttons such as "Submit" to check their progress and "Restart" to begin a new game.





CONCLUSION

The 2D Image Puzzle Game project successfully combines the concepts of image manipulation, interactive design, and JavaScript-based game mechanics to deliver an engaging and challenging user experience. By breaking down a selected image into 100px X100px pieces and utilizing the divide-and-conquer shuffle algorithm, the game presents players with a dynamic puzzle to solve within a set time limit. This project not only demonstrates the ability to work with HTML, CSS, and JavaScript but also highlights creativity in implementing custom animations and responsive design.

Through the use of various features like a countdown timer, move counter, and interactive piece swapping, the game encourages problem-solving and time management skills. Additionally, the game's flexible structure, with randomized image selections and multiple levels of difficulty, ensures a fresh experience for each playthrough.

In summary, this project is a testament to the effective use of web technologies to create a fun and educational puzzle game, showcasing skills in web development, user interface design, and algorithmic logic.

Future Enhancements:

1.This project can be implemented for the image captcha. The interface would be fun and be useful to detect the bot.

2. We would take this project to next level, by building a college puzzle Ridder. The plan is having a college layout in our website, if we click a place of our college then the displaced will be a puzzle of that place, if puzzle is completed within the time, then the reward is the information about the place and some pictures of that particular place.

REFERENCES

1. Introduction to the Design and Analysis of Algorithms by Levitin Anan.

2. Web Programming: Build Internet Applications by Bates. Chris.

3. A Study on the role of Web Technology in Enhancing Research pursuance among university academia. By IRSHAD HUSSAIN and MUHAMMAD (Research papers).